

Introducción a los sistemas en tiempo real

- Introducción
- Clasificación de los sistemas en tiempo real
- Ejemplos de los sistemas en tiempo real
- Características de los sistemas en tiempo real

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

1

1

Definición

- Algunas definiciones de STR (sistema de tiempo real):
 - Oxford Dictionary of Computing:
 - “Cualquier sistema en el que **el tiempo** en el que se produce la salida **es significativo**. Esto generalmente es porque la entrada corresponde a algún movimiento en el **mundo físico**, y la salida está relacionada con dicho movimiento. El intervalo entre el tiempo de entrada y el de salida **debe ser lo suficientemente pequeño para una temporalidad aceptable**”
 - Young (1982):
 - “...cualquier actividad o sistema de proceso de información que tiene que **responder a un estímulo de entrada** generado externamente en **un periodo finito y especificado**”
 - Randell et al. (1995):
 - “Un sistema de tiempo real es aquél al que se le solicita que reaccione a **estímulos del entorno (incluyendo el paso de tiempo físico)** en intervalos del tiempo dictados por el entorno”

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

2

2

Definición

- Resumiendo:
 - Los STR están interaccionan con el mundo real. Responden al paso del tiempo (activados por tiempo) o a sucesos externos (activados por eventos)
 - No sólo es importante el resultado lógico de la computación, sino también **el tiempo en el que se producen los resultados**
 - En la práctica,
 - Las **restricciones temporales** (cuándo se genera un resultado, cuánto tarda en generarse...) forman parte las **especificaciones que debe cumplir el sistema**.
 - El sistema **puede no funcionar correctamente** si las restricciones temporales se incumplen 😞
 - El sistema **será sin duda rechazado** si no se cumplen 😞
 - Además (características ausentes de muchas definiciones pero habitualmente muy importantes):
 - Alta **fiabilidad** (no debería fallar...)
 - **Tolerancia a fallos** (algún componente puede fallar y fallará; hay que tenerlo en cuenta)

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

3

Restricciones temporales

- Ciclo (T)
 - No existe en actividades esporádicas (activadas por eventos)
- Coste (C)
 - Tiempo de ejecución **máximo** (peor) **si no existen interferencias** de otras actividades
 - Difícil de medir con precisión (¿Cuál es el peor de los casos que a la vez sea realista?)
- Tiempo de respuesta (R)
 - Tiempo de ejecución **máximo** (peor) **en condiciones reales** de funcionamiento
 - No puede resultar ser inferior al coste (es imposible...)
- Tiempo límite, plazo o “deadline” (D)
 - Máximo tiempo de respuesta R admisible (**condición de diseño**)
 - Nunca puede fijarse inferior al coste, C (no sería viable...)
- **Restricción temporal típica:**
 - Tiempo de respuesta no superior al plazo: **$R \leq D$**

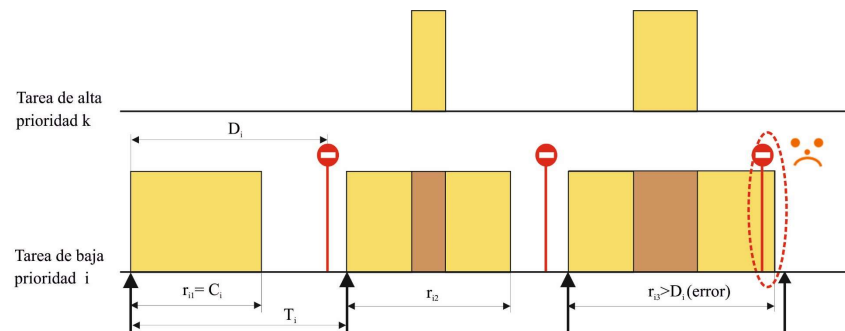
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

4

Restricciones temporales



- $R_i = \max_j(r_{ij})$
- $C_i = \max_j(c_{ij})$
- Las tareas de alta prioridad “roban” tiempo a las de baja prioridad (**interferencia**)

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

5

Clasificación

- Clasificación habitual:
 - Restricciones **críticas** (“hard real time”): El tiempo de respuesta debe ser siempre menor que el máximo – si no es así, será rechazado
 - Restricciones **no críticas** (“soft real time”): Se pueden tolerar retrasos ocasionales dentro de ciertos límites (10% de los ciclos, por ejemplo). Dos subtipos:
 - Restricciones “**firmes**” (“firm”): Si se incumplen, el resultado tardío carece de valor, aunque no se considere un fallo
 - Restricciones “**no-firmes**”: Si se incumplen, el resultado tardío es útil, pero no óptimo
- Un mismo sistema puede tener ambos tipos de restricciones (críticas y no críticas) en funciones distintas
- Algunas funciones pueden tener ambos tipos de restricciones:
 - Restricción no crítica a 50 ms
 - Restricción crítica a 200 ms

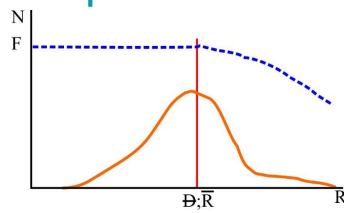
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

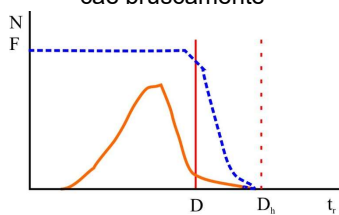
6

Tipos de sistemas y especificaciones

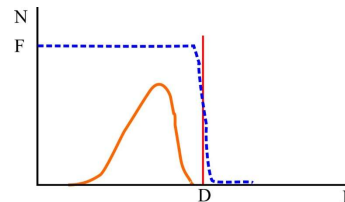


- No de tiempo real:
 - No se definen plazos
 - La calidad del funcionamiento se degrada muy lentamente

- Tiempo real crítico:
 - Plazo estricto
 - La calidad del funcionamiento cae bruscamente



- Tiempo real no crítico:
 - Se toleran incumplimientos del plazo
 - La calidad del funcionamiento cae de manera progresiva
 - Puede haber otro plazo mayor que sí sea crítico



09/09/2025

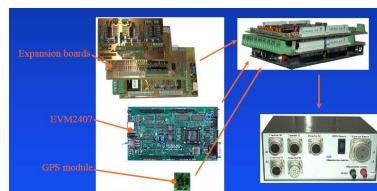
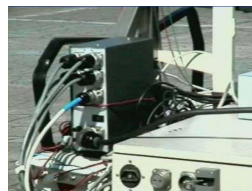
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

7

Sistemas empotrados

- Concepto de sistemas empotrados
 - También conocidos como embebidos, o embarcados
 - STR y sistema empotrado no siempre se consideran sinónimos
 - Funcionan como **un componente más del sistema**
 - No son computadores desde el punto de vista del usuario
 - No tienen los recursos habituales en un computador
 - Tienen los recursos y periféricos apropiados para su función, probablemente no los habituales



09/09/2025

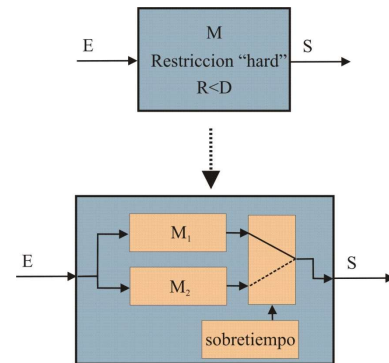
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

8

Combinación de restricciones

- Tenemos dos algoritmos para obtener S a partir de E:
 - M_1 : cumple restricción "firm".
Solución óptima, pero a veces $R_{M1} > D$
 - M_2 : cumple restricción "hard".
Solución subóptima, pero siempre $R_{M2} \ll D$
- Pueden combinarse ambos para conseguir la restricción crítica de M: M_2 se usa sólo en caso de sobretiempo



09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

9

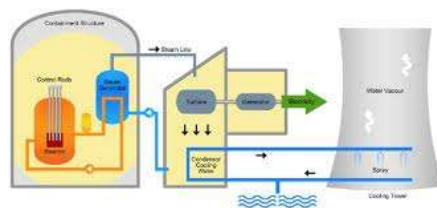
Ejemplos de S.T.R.



Sistema de control de procesos



Sistema de control de producción



Central eléctrica

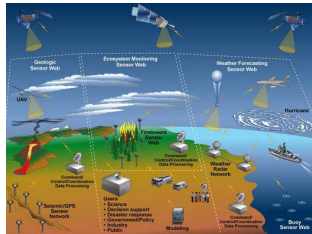
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

10

Ejemplos de S.T.R.



Sistema de mando y control

Sistemas aéreos tripulados y no tripulados



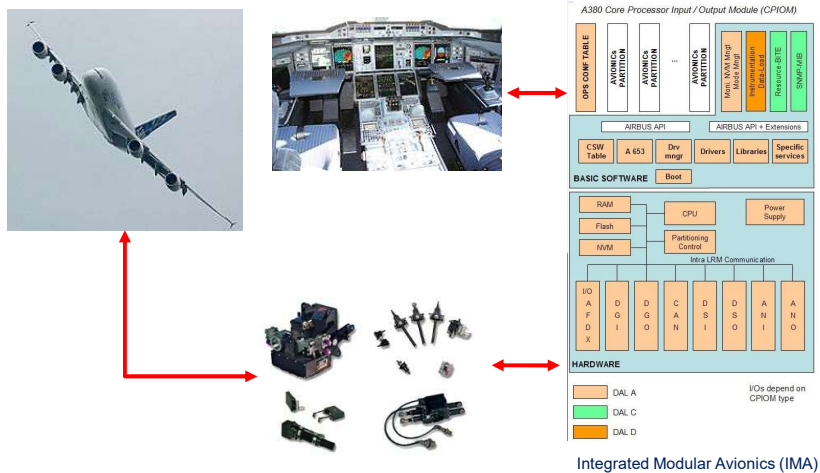
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

11

Ejemplos de S.T.R.



09/09/2025

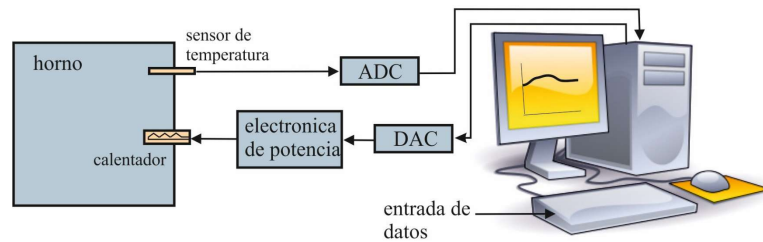
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

12

Un sistema de tiempo real básico

- Función principal: Control de temperatura
- Otras funciones:
 - Mostrar estado del sistema a lo largo del tiempo
 - Leer órdenes de usuario
 - Mantener un reloj de tiempo real



09/09/2025

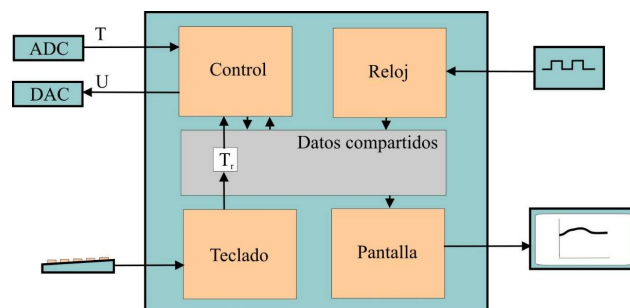
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

13

Un sistema en tiempo real básico

- Implementación con módulos concurrentes
- Necesidad de comunicación y sincronización
- El tiempo de CPU es compartido. Se necesita planificación ("scheduling")



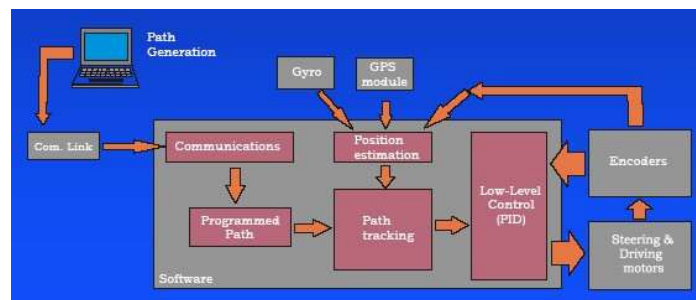
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

14

Controlador de vehículo



09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

15

Ejemplos de S.T.R.



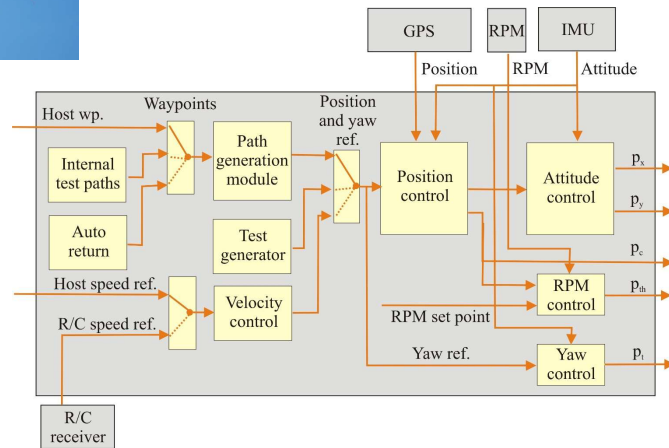
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

16

Ejemplos de S.T.R.



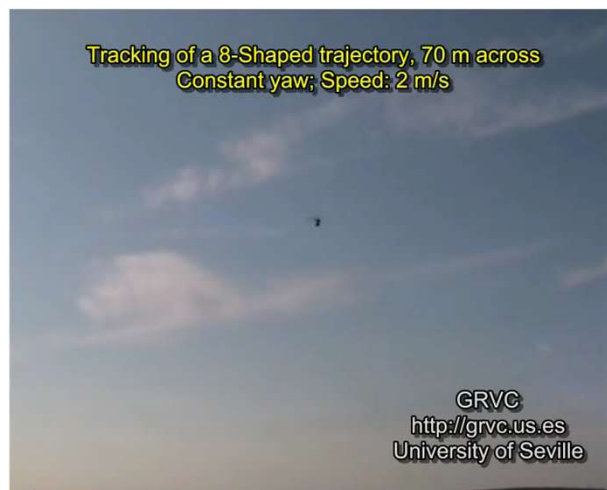
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

17

Ejemplos de S.T.R.



09/09/2025

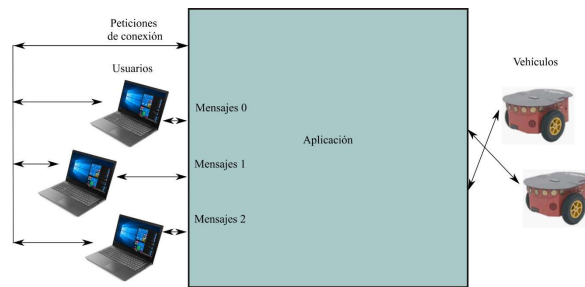
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

18

Un ejemplo de examen (21/1/15)

- Aplicación de control remoto de robots móviles:
 - Hay **N_VEH** robots disponibles
 - Los usuarios se conectan para reservar un robot y poder controlarlo de manera remota (**en exclusiva**, claro...)
 - Cada usuario envía órdenes de movimiento y recibe resultados
- Sistema distribuido
 - Comunicación mediante mensajes



09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

19

Un ejemplo de examen (21/1/15)

- Interacción con el “exterior”:
 - Atender peticiones de conexión
 - Recibir órdenes de los usuarios y responder con resultados
 - Enviar órdenes a los vehículos y esperar recibir resultados
- Tareas concurrentes (en paralelo):
 - Tarea de atención de peticiones de conexión
 - Bucle mientras no llegue la condición de fin de aplicación:
 - Esperar una nueva conexión
 - Crear tarea para atenderla
 - Tareas que atienden una conexión de usuario
 - Reservar vehículo libre (puede implicar espera...)
 - Bucle mientras dure la conexión:
 - Recibir orden del usuario y enviarla al vehículo (posiblemente en otro formato)
 - Esperar resultado del vehículo y enviarlo al usuario

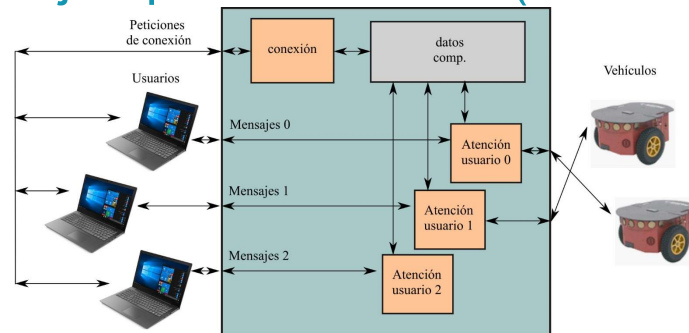
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

20

Un ejemplo de examen (21/1/15)



- ¿Podría hacerse sin concurrencia?
 - Esperar recepción en varios canales a la vez (en ambos lados, usuarios y vehículos)
 - “Seguir la pista” de qué usuario va con qué vehículo, y qué vehículo está pendiente de responder
 - Programación más complicada y menos intuitiva

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

21

Un ejemplo de examen (21/1/15)

- Otras condiciones del problema:
 - **Número máximo de conexiones:** Sólo puede haber más **N_MAX_USU** usuarios conectados a la vez; no se aceptan conexiones mientras se mantenga ese número
 - **Sobretiempo para la espera de vehículo libre:** Una vez conectado, un usuario debe **esperar a que haya vehículo libre**; la espera durará como máximo **t_veh** segundos
 - **Fin de conexión:** Las conexiones acaban al principio, por **sobretiempo** de la espera de robot libre, o bien por iniciativa del usuario (**mensaje de fin**) o
 - **Fin de la aplicación:**
 - Terminación por **orden externa** – antes hay que **esperar a que se desconecten** todos los usuarios
 - No existen usuarios conectados durante **t_conex** segundos seguidos (**sobretiempo**) – evidentemente no hay que esperar a que se desconecten...

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

22

Un ejemplo de examen (21/1/15)

- Una versión más completa de las tareas:
 - Tarea de atención de peticiones de conexión
 - Hacer mientras no fin:
 - Esperar a que el número de conexiones sea inferior al máximo
 - Esperar una nueva conexión
 - Crear tarea para atenderla
 - Tarea que atiende una conexión:
 - Esperar vehículo libre (max. **t_veh** segundos)
 - Si ha habido sobretiempo, cerrar conexión
 - Si no,
 - Hacer mientras dure la conexión:
 - Recibir orden del usuario
 - Si orden de cerrar, cierre de conexión
 - Si no,
 - Enviarla al vehículo (posiblemente en otro formato)
 - Esperar resultado del vehículo y enviarlo al usuario
 - Fin hacer

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

23

23

Un ejemplo de examen (21/1/15)

- Falta resolver el fin de aplicación
- Datos compartidos:
 - Comunicamos las tareas compartiendo datos (estarán en el mismo computador)
 - Hace falta compartir un dato si deben cambiarlo o tenerlo en cuenta diferentes tareas
 - Parece que necesitamos al menos
 - Una **cuenta de conexiones**, porque debemos esperar a que no sea máxima
 - Una variable lógica por cada robot que nos diga si está **libre u ocupado**
- Operaciones sobre los datos compartidos:
 - Inicialización
 - Esperar a reservar vehículo con sobretiempo (**t_veh**)
 - Liberar vehículo
 - Esperar a que la cuenta de conexiones baje del máximo
 - Incrementar o decrementar la cuenta de conexiones
- ¿Quién y cuándo debe usarlas?

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

24

24

Un ejemplo de examen (21/1/15)

- Fin de la aplicación:
 - En general, la aplicación no debe quedarse “colgada” indefinidamente cuando sucede, pero puede pedirse una terminación “ordenada” que lleva algún tiempo
 - Primera causa: Acción externa
 - La tarea de conexiones tiene dos esperas de duración indefinida; debería abandonarlas de algún modo
 - El enunciado pide esperar a que los usuarios salgan de la manera normal (no encuentran robot, o deciden irse...)
 - Añadir “esperar número de conexiones cero” a las operaciones con datos compartidos
 - Segunda causa: Tiempo máximo sin usuarios conectados
 - Necesitamos “alguien” que cuente el tiempo transcurrido desde que salieron todos los usuarios y nos avise si llega al máximo
 - Ese “alguien” suele ser un servicio del sistema operativo y llamarse **“temporizador”**
 - Podemos arrancarlo, pararlo o esperar a que “avise” (como un despertador..) 
 - ¿Quién y cuando lo arranca, para o espera?
 - Igual que antes, el “aviso” deberá terminar de algún modo las esperas de la tarea de conexiones

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

25

25

Características de los sistemas en tiempo real

- Tiempo de respuesta predecible
- Sistemas grandes y complejos
- Necesidad de soporte para cálculo numérico
- Alta fiabilidad y tolerancia a fallos
- Actividad concurrente de los componentes
- Necesidad de servicios de temporización
- Interacción con el soporte físico (“hardware”)
- Eficiencia frente a predictibilidad

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

26

26

Características de los sistemas de tiempo real

- **Sistemas grandes y complejos**
 - El sistema no debe hacerse de manera “artesanal”
 - Diseño basado en especificaciones
 - ¿Qué debe hacer el código? (con precisión...)
 - Especificaciones funcionales; especificaciones de tiempo de respuesta
 - Descomponer el problema:
 - Descomposición en módulos y abstracción
 - Especificación de funciones de los componentes
 - Especificación de interfaces
 - Extensibilidad: Necesitaremos modificar y ampliar el sistema informático, porque cambia el sistema real
 - ¿Qué es recomendable usar (al menos en teoría)?
 - Lenguajes con soporte para programación orientada a objeto (C++, Java, ADA...) (creación de módulos, ocultación de datos, interfaces...)
 - Sistemas operativos que permitan la ejecución concurrente de módulos (procesos, hilos...)

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

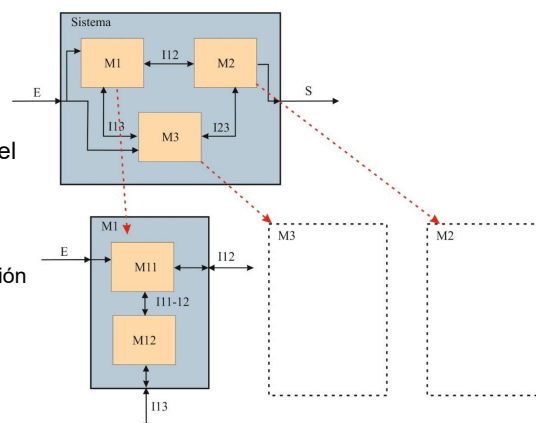
27

27

Características de los sistemas de tiempo real

Descomposición en módulos:

- ¿Qué hace el módulo? (especificación)
- ¿Cómo se comunica con el resto? (interfaces)
- Es posible
 - Desarrollarlos independientemente
 - Cambiar la implementación de cualquiera de ellos



09/09/2025

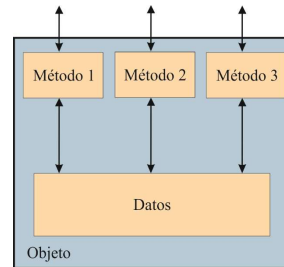
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

28

Características de los sistemas de tiempo real

- Programación orientada a objeto:
 - Objetos que “encapsulan” datos y procedimientos
 - Datos inaccesibles desde el exterior (“ocultación de datos”)
 - Debemos usar procedimientos que modifican los datos (“métodos”)
 - Los objetos permiten implementar módulos; los métodos serán las interfaces



09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

29

29

Características de los sistemas de tiempo real

- Necesidad de soporte para cálculo numérico
 - Es necesario un lenguaje con soporte adecuado para entero y punto flotante
 - Selección del tipo adecuado
 - Extensión (rango de valores representables)
 - Resolución (en punto flotante, intervalo mínimo entre valores representables)
 - Si la representación no es adecuada, tendremos problemas
 - C: Seleccionar un tipo entre varias opciones
 - ¿Cuál es el máximo entero que cabe en un **long**? No se sabe...
 - Tipos enteros con tamaño y extensión conocidos: `int32_t`, `uint16_t`... (`<stdint.h>`, `<inttypes.h>`)
 - ADA: Especificación directa de extensión o resolución
 - Si algo falla, detección y tratamiento de excepciones
 - Aritmética de punto fijo:
 - Números fraccionarios codificados en formato entero
 - Mucho más eficiente si no hay “hardware” de punto flotante
 - ADA: Incluida en el lenguaje. No es necesaria programación “artesanal”

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

30

30

Características de los sistemas en tiempo real

- Alta fiabilidad y tolerancia a fallos
 - Diseño basado en especificaciones para conseguir alta fiabilidad
 - Ante todo, intentar que funcione perfectamente, lo cual es imposible...
 - Sistema tolerante a fallos para superar fallos y circunstancias excepcionales – necesitamos **redundancia**
 - **Redundancia estática**: Varias versiones simultáneas de la misma función
 - **Redundancia dinámica**: Detección de fallos y ejecución de algoritmos de recuperación (**reaccionar** ante el fallo)
 - Los fallos sistemáticos requieren **diversidad** de diseño para que la redundancia sea efectiva
 - Características deseables del lenguaje de programación
 - Facilitar la ausencia de fallos:
 - Claridad
 - Restricciones estrictas para conversiones de tipo
 - Incorporar estructuras de programación útiles, tales como el tratamiento de excepciones (C++, ADA...)

09/09/2025

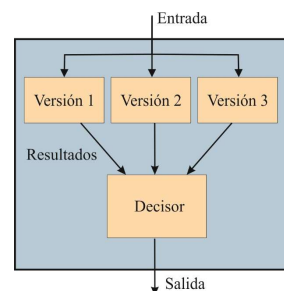
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

31

31

Características de los sistemas de tiempo real

- Redundancia estática (N-versiones):
 - Varias versiones de código en paralelo
 - Se generan N resultados
 - Un módulo adicional genera la salida (por ejemplo, por votación)
 - No siempre es fácil comparar resultados
 - Normalmente existirá también redundancia “hardware” (diferentes computadores)
 - Necesita N veces más recursos



09/09/2025

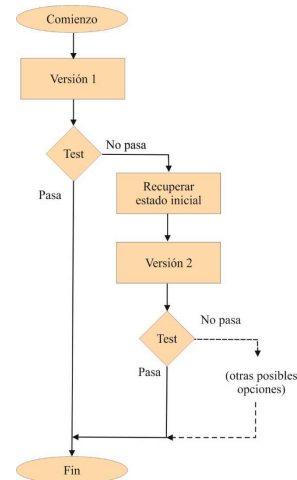
© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

32

32

Características de los sistemas de tiempo real

- Redundancia dinámica en el caso general:
 - Detectar **error** (**estado incorrecto**)
 - Valorar daños (¿qué es erróneo?)
 - Recuperar error (llevar el módulo a un estado correcto)
 - Tratar fallo (intentar eliminar el **fallo**, es decir, la **causa del error**)
- Bloques de recuperación (caso particular):
 - Test de aceptación (detección de errores)
 - Ante error, usar versiones alternativas del algoritmo (**no se ejecutan siempre**; sólo cuando hay error)
 - Pueden implementarse usando tratamiento de excepciones (C++, Java, ADA)



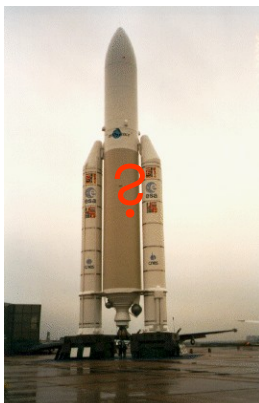
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

33

33

Fallo del vuelo 501 del Ariane 5



- Vuelo normal hasta el segundo 36
- Maniobra brusca a los 37 s
- Explosión a unos 4000 m
- Restos esparcidos en 12 Km² de pantanos
- 4 satélites perdidos

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

34

34

Fallo del vuelo 501 del Ariane 5



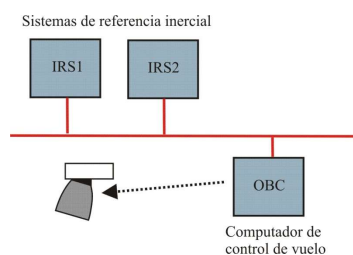
09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

35

35

Fallo del vuelo 501 del Ariane 5 (4/6/96)

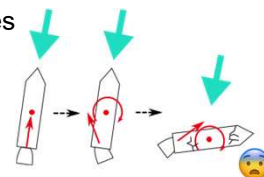


• Causas:

- Cambio del perfil de vuelo
- Error aritmético no tratado
- Redundancia no efectiva
- Función inútil activa en vuelo
- Falta de especificación y prueba

Secuencia de fallo:

- Error de conversión de 64 a 16 bits en función de autocalibración
- Falla el IRS activo
- !No puede actuar el de respaldo porque falló pocos ms antes!
- El control de vuelo recibe datos espúreos
- Tobera en ángulo límite
- Vehículo en ángulo extremo
- Daños estructurales
- Autodestrucción



09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

36

36

Características de los sistemas en tiempo real

- **Actividad concurrente de los componentes**
 - Habitualmente la implementación más natural
 - Se necesita sistema operativo multitarea o sistema de soporte en tiempo de ejecución
 - Necesidad de utilizar servicios de comunicación, sincronización y planificación
 - Dos opciones de implementación
 - Sistema operativo + lenguaje de programación convencional (C, C++)
 - Lenguajes de programación concurrentes (ADA, Java)
- **Servicios de temporización**
 - Se necesita una base de tiempos: Reloj de tiempo real. Puede medir intervalos o “dar la hora”
 - Servicios específicos:
 - Activación periódica (“activar cada 10 ms...”)
 - Retraso relativo (“esperar 100 ms”)
 - Activación en un instante determinado (fecha, hora)
 - Sobretiempo: Capacidad para abortar funciones o esperas (“Esperar llegada de mensaje durante 100 ms como máximo”)

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

37

37

Características de los sistemas en tiempo real

- **Interacción con el soporte físico (“hardware”)**
 - Puede ser necesario gestionar el soporte físico a bajo nivel
 - Dispositivos especiales
 - Maneras especiales de gestionar dispositivos convencionales
 - El “software” de entrada/salida de bajo nivel es en sí mismo un ejemplo de sistema de tiempo real
 - El “software” de bajo nivel debería codificarse (en su mayor parte) mediante lenguajes de alto nivel
 - Compromiso entre fiabilidad y flexibilidad del lenguaje
 - Falta de portabilidad (Excepción: ADA)
- **Eficiencia frente a predictibilidad**
 - La eficiencia **no es un fin en sí misma**
 - Lo fundamental es que el sistema sea **predecible**: Deben cumplirse los plazos (**sin excepciones**, si la restricción es crítica)
 - Puede que sea necesaria la eficiencia **como medio** para tal fin
 - Es necesario evitar funciones de coste no predecible dentro de actividades críticas:
 - Arrancar actividades concurrentes
 - Pedir memoria
 - ...

09/09/2025

© Joaquín Ferruz Melero 2025-26 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

38

38